A Pentium M Style Hybrid Branch Predictor with a Two-Way Global Table

S M Masrur Ahmed
Assignment 1: Branch Predictor Implementation
ID: 2393372
Email: sahmed75@uh.edu

Abstract

I implemented a simplified version of the Intel Pentium M branch outcome predictor for this assignment. I combined a bimodal table with a tagged, set-associative global table and used the branch address and global history to index it. I chose the two-way global table allowed by the assignment. I built the predictor in the CBP2 framework and tested it on all 20 supplied traces. My predictor gets 8.660 mispredictions per 1000 instructions (MPKI) on average, compared to 10.267 for the bimodal table alone. I then tried two changes to how I allocate global entries. With those changes, the same table sizes give 6.134 MPKI, better than the 8 KB gshare predictor supplied with the framework.

1 Introduction

When a processor reaches a conditional branch, it has to choose which instructions to fetch before it knows the branch outcome. A correct prediction lets it keep working. A wrong prediction means it must discard work from the wrong path. In this assignment, I tested how a predictor that learns from earlier branch outcomes can reduce those mistakes.

The Pentium M was designed for notebooks, so its branch predictor had to support both performance and lower power use. I followed the organization that Uzelac and Milenković reconstructed using microbenchmarks [2]. Milenković et al. used a similar approach for the Pentium III and Pentium 4 [3]. For my implementation, I used the block diagram in the assignment [1] and focused on the bimodal and global outcome predictors. I used two ways in the global table rather than the four ways of the real processor, as the assignment allows.

I found the following:

- My predictor gets 8.660 MPKI on average over the 20 CBP2 traces, which is 15.6% fewer mispredictions than the bimodal table alone (10.267 MPKI).
- I matched all 22 rows of the worked example in the assignment.
- With the assignment's allocation rule, I got worse results on 10 of the 20 traces after adding the global table. I reduced most of that loss by allocating only when the bimodal table was wrong and resetting each new entry's counter. Together, these changes give 6.134 MPKI.

I followed a seven-section structure similar to [3]. In Section 2, I cover the predictor background. I describe the task in Section 3 and my test setup in Section 4. In Section 5, I explain the code and experiments. I discuss the results in Section 6 and finish with what I learned from them.

2 Dynamic Branch Prediction

I used 2-bit saturating counters, a common building block for dynamic predictors [4]. Each counter increases on a taken branch and decreases on a not-taken branch. It predicts taken at 2 or 3. From strongly taken (3) or strongly not taken (0), two opposite outcomes in a row are needed to reverse the prediction. From a weak state, one is enough. This helps with a loop branch that is taken repeatedly and then falls through once, since that one exit does not reverse a strongly taken prediction.

A bimodal predictor uses the branch address to select a counter. It can learn a branch's usual direction, but it does not record the outcomes of earlier branches. Two-level predictors add that history [5, 6]. A global predictor keeps recent outcomes in a shared shift register. The gshare example in the CBP2 framework XORs this history with the branch address to select a counter [7]. In a hybrid predictor, I can combine predictors and select which prediction to use [7].

The Pentium M combines a 4096-entry bimodal table indexed by `IP[11:0]`, a loop predictor, and a global predictor with 512 sets and four ways [2]. It hashes the branch address with a path information register to access the global table, where each entry has a 6-bit tag. I kept the same selection rule in my version. I use the bimodal prediction unless the global table has a matching valid tag. On a hit, I use the global prediction. Figure 1 shows the part I implemented.

3 Problem Statement

My task was to implement the member functions of `pm_predictor` in `my_predictor.h`. I needed the bimodal table and global predictor for branch outcomes. I left out the loop predictor, branch target prediction (BTB and indirect

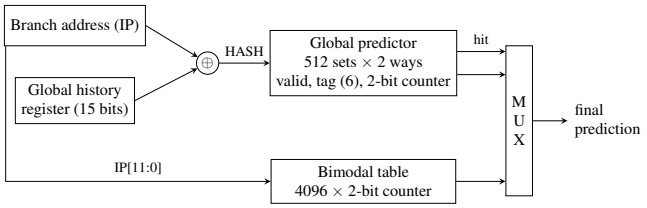


Figure 1: The part of the Pentium M predictor I implemented. The index is HASH[14:6] and the tag is HASH[5:0]. On a global hit the multiplexer selects the global prediction.

BTB), and path information register with its hash, since the assignment does not require them. I also chose the permitted two-way global table instead of four ways.

I used the implementation to answer four questions about the global table:

1. Does my implementation behave exactly like the worked example in the assignment?
2. How much does the two-way global table improve on the bimodal table alone, and how does it compare to the gshare predictor of the framework?
3. How much is lost by using two ways instead of four?
4. Does the way new entries are allocated in the global table matter?

4 Experimental Environment

I used the CBP2 branch prediction infrastructure by Jiménez [8]. Its driver reads a trace, calls `predict` for every branch and then `update` with the real outcome. It only counts the direction mispredictions of conditional branches, and it reports them as mispredictions per 1000 instructions (MPKI). The framework comes with 20 traces from the SPEC CPU2000 integer and SPECjvm98 benchmarks. Each trace has 100 million instructions.

I built the predictor with `make`, which runs `g++ -O3`. On my macOS machine, that compiler is Apple clang 21. I ran the traces with `run traces`. I also wrote `experiments/sweep.cc` to compare configurations without reading each trace repeatedly. It runs several predictors side by side using different constructor parameters for the same `pm_predictor` class.

5 Implementation and Experiments

5.1 Tables

I gave the bimodal table 4096 2-bit counters indexed by `IP[11:0]` and initialized them to 2, or weakly taken. I used 512 sets with two ways in the global table. Each way has a valid bit, a 6-bit tag and a 2-bit counter. The hardware model needs one LRU bit per set. I made the sizes constructor parameters with `#define` defaults so I could also run the small example from the assignment with the same class:

```
#define PM_BIMODAL_BITS 12      // 4096-entry bimodal table
#define PM_INDEX_BITS   9       // 512 sets in the global
     predictor
#define PM_TAG_BITS     6       // 6-bit partial tags
#define PM_WAYS         2       // 2-way global predictor (the
     assignment allows 2 instead of 4)
#define PM_ADDR_SHIFT   0       // IP bits feeding the hash start
      at bit 0
// what to do with the 2-bit counter of a newly allocated global
     entry
#define PM_ALLOC_RESET  0       // set it to weakly taken /
     weakly not taken
#define PM_ALLOC_KEEP   1       // keep the old counter value and
      train it (handout example)
// allocation policy defaults; the handout example uses
     PM_ALLOC_KEEP and
// allocates on every global miss
#define PM_ALLOC_COUNTER                PM_ALLOC_KEEP
#define PM_ALLOC_ON_BIMODAL_MISS        false
```

5.2 Hash, index and tag

I used a 15-bit global history register (GHR) instead of the optional path information register. It holds the outcomes of the last 15 conditional branches. The hash is `IP[14:0] XOR GHR`. The set index is `HASH[14:6]` and the tag is `HASH[5:0]`, like in the diagram. This is the same idea as the 4-bit example in the assignment, where the address bits are XORed with a 4-bit GHR. I used the low address bits starting at bit 0, because x86 instructions have different lengths and the low bits of the address are useful.

5.3 Update and replacement

I train the bimodal counter after every conditional branch, even when I used the global prediction. On a global hit, I train the matching way's counter and mark that way as most recently used. On a miss, I choose an invalid way if one is available. Otherwise, I replace the least recently used way. I write the new tag, keep the way's old counter value, and train it with the actual outcome, following the worked example. I then shift the outcome into the GHR. Unconditional branches do not change these tables or the history. Figure 2 shows my `predict` and `update` functions.

5.4 Checking against the worked example

Before running the traces, I checked my predictions against Appendix B. The example uses 8-bit addresses, a 16-entry bimodal table, a two-way global table with 4 sets and 2-bit tags, and a 4-bit GHR. I wrote `tests/example_test.cc` to run my class with those sizes and the handout's initial counter values. I replayed the 22 branches and checked the local prediction, global hit and prediction, and final prediction on each row. Figure 3 shows the output. All 22 rows match the handout.

5.5 Experiments

Besides the submitted predictor, I ran these configurations on all 20 traces:

```
branch_update *predict (branch_info & b) {
    bi = b;
    if (b.br_flags & BR_CONDITIONAL) {
        // bimodal (local) prediction
        u.index = b.address & ((1 << bimodal_bits) - 1);
        u.bimodal_prediction = bimodal[u.index] >> 1;

        // global prediction: hash the IP with the global history
        int hash_bits = index_bits + tag_bits;
        unsigned int hash = ((b.address >> addr_shift) ^ history)
     & ((1 << hash_bits) - 1);
        u.set = hash >> tag_bits;
        u.tag = hash & ((1 << tag_bits) - 1);
        u.way = -1;
        for (int w = 0; w < ways; w++) {
            int e = u.set * ways + w;
            if (valid[e] && tags[e] == u.tag) {
                u.way = w;
                break;
            }
        }

        // global hit overrides the bimodal prediction
        if (u.way >= 0)
            u.direction_prediction (ctr[u.set * ways + u.way] >>
     1);
        else
            u.direction_prediction (u.bimodal_prediction);
    } else {
        u.direction_prediction (true);
    }

    u.target_prediction (0);
    return &u;
}
```

```
void update (branch_update *bu, bool taken, unsigned int target) {
    if (!(bi.br_flags & BR_CONDITIONAL)) return;
    pm_update *pu = (pm_update *) bu;

    // the bimodal table is always trained
    train (&bimodal[pu->index], taken);

    if (pu->way >= 0) {
        train (&ctr[pu->set * ways + pu->way], taken);
        touch (pu->set, pu->way);
    } else if (!alloc_on_bimodal_miss || pu->bimodal_prediction !=
       taken) {
        // global miss: allocate an invalid way, or else the LRU
      way
        int victim = 0;
        for (int w = 0; w < ways; w++) {
            int e = pu->set * ways + w;
            if (!valid[e]) { victim = w; break; }
            if (lru[e] > lru[pu->set * ways + victim]) victim = w;
        }
        int e = pu->set * ways + victim;
        valid[e] = 1;
        tags[e] = pu->tag;
        if (alloc_counter == PM_ALLOC_RESET)
            ctr[e] = taken ? 2 : 1;
        else
            train (&ctr[e], taken);
        touch (pu->set, victim);
    }

    // shift the outcome into the global history register
    history = ((history << 1) | taken) & ((1 << (index_bits +
      tag_bits)) - 1);
}
```

Figure 2: The `predict` (left) and `update` (right) functions of my `pm_predictor` class.

```
row addr  outcome local global final  check
  1 0x44  T       N     -      N      ok
  2 0x90  T       N     -      N      ok
  3 0x44  NT      T     -      T      ok
  4 0x6C  NT      T     -      T      ok
  5 0x90  T       N     -      N      ok
  6 0x44  T       N     T      T      ok
  7 0x90  T       T     -      T      ok
  8 0x44  NT      T     -      T      ok
  9 0x6C  NT      T     -      T      ok
 10 0x90  T       T     T      T      ok
 11 0x44  NT      N     T      T      ok
 12 0x6C  T       N     -      N      ok
 13 0x44  NT      N     -      N      ok
 14 0x6C  NT      T     N      N      ok
 15 0x90  T       T     -      T      ok
 16 0x44  T       N     T      T      ok
 17 0x90  T       T     -      T      ok
 18 0x44  NT      N     -      N      ok
 19 0x6C  NT      N     -      N      ok
 20 0x90  T       T     T      T      ok
 21 0x44  T       N     T      T      ok
 22 0x90  T       T     -      T      ok
22/22 rows match the handout, 13/22 final predictions correct
```

Figure 3: Output of my test for the worked example of the assignment ("-" means no global hit).

- I tested the bimodal table alone, the framework's gshare predictor, and always taken.
- I tested the same predictor with 1 and 4 ways. I kept 512 sets, so the global capacity changed to 512, 1024 or 2048 entries along with the associativity.
- I used `IP[18:4]` instead of `IP[14:0]` in the hash.

Table 1: Average MPKI over the 20 traces. Storage is estimated for packed hardware tables.

Predictor	Est. HW storage	Avg. MPKI
Always taken	0	86.312
Bimodal only	1 KB	10.267
gshare (framework)	8 KB	6.306
PM, 2-way (mine)	≈2.2 KB	**8.661**
PM, 2-way, improved alloc.	≈2.2 KB	6.134
PM, 4-way	≈3.75 KB	7.758

- I tried resetting new counters to weakly taken or weakly not taken based on the outcome. I also tried allocating only when the bimodal prediction was wrong.
- I combined both allocation changes with 2 and 4 ways.

6 Results

6.1 Overall results

I got an average of 8.660 MPKI from the official `run traces` script. In the tables, I report 8.661 because I round the average of the printed per-trace values. The script truncates that average. Table 1 compares my predictor with the alternatives.

When I added the two-way global table, the average dropped from 10.267 to 8.661 MPKI, which is 15.6% fewer

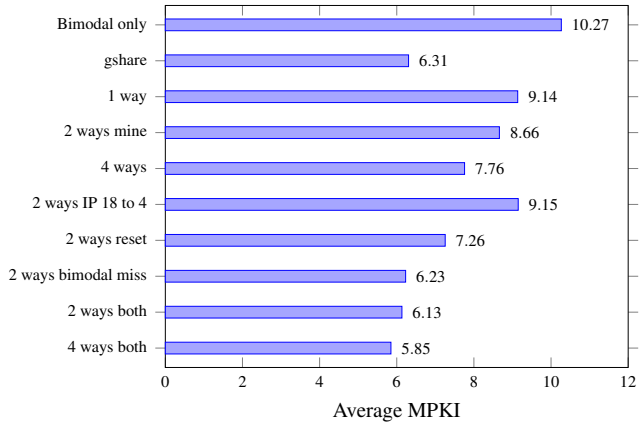


Figure 4: Average MPKI of the predictors and configurations from Tables 1 and 3.

mispredictions. I saw the largest gains on eon, from 9.443 to 3.838, jess, from 8.773 to 2.554, and mcf, from 33.271 to 20.081. These gains suggest that history helps on those traces. I still got a better average with gshare (6.306), but its hardware tables would need about 8 KB, almost four times as much storage.

6.2 Results per trace

I list all 20 traces in Table 2. Adding the two-way global table made 10 of the 20 traces worse than the bimodal table alone: vpr, gcc, raytrace, javac, mpegaudio, mtrt, perlbmk, vortex, bzip2 and twolf. For example, twolf goes from 21.943 to 24.239 MPKI.

I think the allocation rule explains much of this loss. In my default version, I allocate an entry on every global miss, including branches that the bimodal table already gets right. With only 1024 entries and two ways per set, those allocations can replace useful entries. A later hit can also use a counter trained by a different branch and override a correct bimodal prediction. I tested changes to this rule in the next experiment.

6.3 Associativity and allocation

Table 3 and Figure 4 show the average MPKI for all the configurations I tried.

When I increased the table from 1 to 2 to 4 ways, the average fell from 9.136 to 8.661 to 7.758 MPKI. I kept 512 sets, so the table also grew to 512, 1024 and 2048 entries. I therefore cannot separate the benefit of more ways from the benefit of more capacity in this experiment. My two-way version loses 0.90 MPKI compared with four ways but needs about half the global table storage. When I used `IP[18:4]` in the hash, the result was 5.6% worse. For these x86 traces, keeping the low address bits worked better.

I got the largest improvement by changing allocation. When I allocated only after a wrong bimodal prediction, the average fell by 28.1% to 6.230 MPKI. Resetting each new entry's counter helped too, with a 16.2% reduction. That avoids starting a new branch with the previous branch's counter bias. Together, the two changes gave me 6.134 MPKI, or 40.3% fewer mispredictions than bimodal alone. This is better than gshare with about a quarter of its hardware storage, and only 0.28 MPKI behind four ways with the same changes (5.849). I improved on bimodal in 19 of the 20 traces. The exception is twolf, which remains 0.04 MPKI worse.

I still got more than 10 MPKI on gzip, vpr, gcc, mcf and twolf after changing allocation. I would need more measurements to tell how much of that comes from table capacity or branches whose outcomes a 4K-entry bimodal table and a 15-bit history cannot predict well.

6.4 Storage

I estimated hardware storage with packed bits. The bimodal table needs $4096 \times 2 = 8192$ bits (1 KB). The two-way global table needs $1024 \times (1+6+2) = 9216$ bits plus 512 LRU bits, so 9728 bits (about 1.2 KB), and the whole predictor about 2.2 KB. The four-way version needs about 2.75 KB for the global table, and the gshare predictor needs 32768×2 bits = 8 KB.

In my C++ code, I used a byte for each counter, tag, valid bit and LRU rank instead of packing the fields. My two-way predictor's arrays therefore take $4096 + 4 \times 1024 = 8192$ bytes (8 KiB). I used this simpler representation because it produces the same predictions.

7 Conclusion

I built the bimodal and tagged two-way global predictors in the CBP2 framework and matched all 22 rows of the assignment's worked example. My default version gets 8.660 MPKI on the 20 CBP2 traces, which is 15.6% better than bimodal alone. Of the changes I tested, allocation made a bigger difference than adding ways. The default allocation rule hurts 10 of the 20 traces. Allocating only after a bimodal mistake and resetting new counters brings the average to 6.134 MPKI without increasing the two-way table sizes.

If I extend this work, I would try the loop predictor and replace the outcome history with the Pentium M's path information register. I would also add branch target prediction, which I left out because it was not required for this assignment.

Table 2: MPKI for every trace. PM = Pentium M hybrid predictor.

Trace	Bimodal only	gshare	**PM 2-way (mine)**	PM 2-way, improved	PM 4-way
164.gzip	15.031	12.473	**12.951**	12.521	12.692
175.vpr	14.673	13.415	**16.823**	13.867	15.967
176.gcc	19.313	11.254	**19.786**	10.605	17.999
181.mcf	33.271	15.837	**20.081**	16.402	18.269
186.crafty	13.097	5.837	**10.818**	5.987	9.105
197.parser	15.420	10.008	**12.615**	9.040	10.933
201.compress	9.368	7.831	**8.635**	7.840	8.017
202.jess	8.773	1.562	**2.554**	1.137	2.000
205.raytrace	3.704	2.756	**4.025**	2.483	3.597
209.db	6.437	3.909	**5.268**	3.738	4.824
213.javac	3.198	2.267	**3.305**	2.139	3.008
222.mpegaudio	3.166	2.188	**3.264**	2.134	2.940
227.mtrt	3.682	2.657	**3.886**	2.467	3.436
228.jack	7.175	3.033	**5.669**	2.422	4.238
252.eon	9.443	1.807	**3.838**	1.530	2.148
253.perlbmk	5.848	2.554	**7.315**	1.855	5.084
254.gap	9.760	3.926	**5.762**	3.607	4.333
255.vortex	1.930	1.222	**2.239**	0.859	1.641
256.bzip2	0.113	0.094	**0.146**	0.064	0.112
300.twolf	21.943	21.489	**24.239**	21.982	24.817
Average	10.267	6.306	**8.661**	6.134	7.758

Table 3: Average MPKI of the configurations I tried, and the change compared to my submitted predictor.

Configuration	Avg. MPKI	Change
1 way	9.136	+5.5%
2 ways (mine)	8.661	–
4 ways	7.758	-10.4%
2 ways, hash uses IP[18:4]	9.149	+5.6%
2 ways, reset new counters	7.256	-16.2%
2 ways, allocate on bimodal miss	6.230	-28.1%
2 ways, both changes	6.134	-29.2%
4 ways, both changes	5.849	-32.5%

References

[1] “Assignment 1: Branch Predictor Implementation,” course handout, including the worked example in Appendix B.

[2] V. Uzelac and A. Milenković, “Experiment Flows and Microbenchmarks for Reverse Engineering of Branch Predictor Structures,” in Proc. IEEE Int. Symp. Performance Analysis of Systems and Software (ISPASS), 2009, pp. 207–217.

[3] M. Milenković, A. Milenković, and J. Kulick, “Demystifying Intel Branch Predictors,” in Proc. Workshop on Duplicating, Deconstructing, and Debunking (WDDD), 2002, pp. 52–61.

[4] J. E. Smith, “A Study of Branch Prediction Strategies,” in Proc. 8th Int. Symp. Computer Architecture (ISCA), 1981, pp. 135–148.

[5] T.-Y. Yeh and Y. N. Patt, “Two-Level Adaptive Training Branch Prediction,” in Proc. 24th Int. Symp. Microarchitecture (MICRO), 1991, pp. 51–61.

[6] S.-T. Pan, K. So, and J. T. Rahmeh, “Improving the Accuracy of Dynamic Branch Prediction Using Branch Correlation,” in Proc. 5th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS), 1992, pp. 76–84.

[7] S. McFarling, “Combining Branch Predictors,” Digital Equipment Corporation, Western Research Laboratory, Technical Note TN-36, June 1993.

[8] D. A. Jiménez, “Infrastructure for Branch Prediction Competition,” CBP2 infrastructure documentation, `cbp2-infrastructure-v2`.